LocalIntent: Outcome-Specific Structured Generation for Backend-Compatible Smart Building Automation with Local LLMs

Tianyang Liu[1], Jiayue Wang[1], Zhanhao Cheng[1], Changyang Wang[1], Shaoshu Zhu[2], Zhenghao Wu[2], and Hadi Tabatabaee Malazi[1,2]

[1] Beijing Dublin International College, University College Dublin, Beijing, China
[2] School of Computer Science, University College Dublin, Dublin, Ireland
Hadi.TabatabaeeMalazi@ucd.ie

Abstract. Locally deployed large language models (LLMs) can reduce reliance on remote cloud services for natural language smart building automation, but backend-facing processing requires correct outcome selection, alignment with local capabilities, and valid structured outputs. We investigate whether a complete outcome-specific prompting design improves compact-model outputs beyond generic route-then-generate prompting. The proposed *LocalIntent* approach routes requests to ordinary or persistent-rule branches: the former produces commands, clarifications, or rejections, while the latter generates trigger–action rules, followed by deterministic validation. We evaluate four locally executed Gemma and Qwen models spanning 2B–9B parameters on 101 non-persistent and 54 persistent-rule records, comparing *Monolithic* prompting, generic two-stage prompting, LocalIntent, and validation-guided repair. Across the evaluated datasets, *LocalIntent* achieves higher intent understanding, device identification, and action/parameter correctness than generic two-stage prompting, while clarification/mode correctness remains model-dependent. The largest gains occur for rules, where *LocalIntent* improves intent understanding from 26.4% to 80.6%, device identification from 59.3% to 84.7%, rule-mode correctness from 49.1% to 98.6%, and action/parameter correctness from 5.6% to 44.9%. For non-persistent requests, pooled gains are smaller: intent understanding increases from 39.6% to 53.2%, and action/parameter correctness from 30.7% to 42.8%.

Keywords: Smart building automation · Local LLM inference · Edge intelligence · Intent understanding · Structured output generation · Automation rule generation

1 Introduction

Smart buildings coordinate lighting, ventilation, blinds, and sensor-driven services. Natural language interfaces can improve access, but requests require different backend outcomes: *"turn on the meeting-room lights"* is an immediate command, whereas *"turn on the meeting-room lights whenever occupancy is detected"* defines a persistent rule. Incomplete requests require clarification; unsupported requests require rejection.

Intent processing must select the requested behaviour and backend outcome, then generate a representation the platform can process. *Automation-output correctness* requires correct outcome selection, schema compliance, grounding in available devices

and sensors, supported actions and parameters, and semantic consistency. A well-formed output is insufficient if it invents capabilities, guesses missing information, or executes future-triggered behaviour immediately.

These requirements are particularly important for local LLMs, which must process requests near the managed environment while relying on local capability information and limited computational resources. On-device LLMs have supported Home Assistant intent detection and response generation [3] and privacy-oriented personalised assistance [7]. However, local processing does not ensure backend-ready output: compact models must still interpret temporal scope, select an outcome, resolve entities, and generate structured output within resource and response-time constraints.

Prior work examines LLM-based device control [10], IoT program synthesis [18,17], rule authoring [6], and structured generation [12,19]. Evaluations also expose failures on invalid, ambiguous, and multi-device requests [11]. These strands usually treat intent interpretation, program construction, rule authoring, or schema compliance separately. They provide limited evidence on the complete backend-facing transformation from a request to a command, stored rule, clarification, or rejection, followed by a representation valid for that outcome. Existing approaches rarely evaluate whether outcome selection itself should determine the generation constraints and validation criteria used for backend-facing automation. Our hypothesis is that the complete LocalIntent design achieves higher automation-output correctness than generic route-then-generate prompting, particularly for persistent rules.

We introduce *LocalIntent*, which routes requests to an ordinary or persistent-rule branch. The rule branch generates a predicate-bearing trigger and action. The ordinary branch produces a target–action–parameter command, clarification, or rejection under non-execution guardrails. A deterministic validator tests structure, registered entities, supported actions and parameters, and mechanically testable rule consistency. We compare *LocalIntent* with *Monolithic* prompting and *TwoStageGeneric*, a generic route-then-generate design. The methods differ in routing labels, prompts, schemas, context presentation, and fallbacks; thus, we compare complete designs without isolating individual mechanisms. We also assess validation-guided repair and local inference latency. *LocalIntent*'s clearest differences occur on persistent rules; non-persistent differences are smaller and model-dependent, and repair adds little aggregate benefit.

This paper makes four contributions:

1. We formulate local LLM-based smart building automation as structured generation with four outcomes: immediate commands, persistent rules, clarification, and rejection, each with distinct temporal semantics and validity conditions.
2. We introduce *LocalIntent* with ordinary/rule routing, command and non-execution guardrails, persistent-rule mapping, and automatic local-capability validation.
3. We construct two controlled evaluation datasets: 101 non-persistent requests covering commands, clarifications, and rejections, and 54 persistent-rule requests. We evaluate four local Gemma and Qwen models (2B–9B parameters).
4. We release implementation and evaluation artefacts in a public repository[3].

[3] https://github.com/ucd-soc2/localIntent

2 Related Work

Natural-language smart-device interfaces may initiate physical actions, making ambiguous or unsupported requests consequential. Sasha [10] translates underspecified goals into device plans but reports unsuitable and hallucinated behaviour. HomeBench [11] exposes difficulties with valid and invalid, single- and multi-device instructions, while Calò et al. [4] use multimodal information to resolve ambiguity. For local deployment, these safety concerns coexist with privacy and resource constraints. At the infrastructure level, distributed service placement and workload orchestration address response-time and backhaul-traffic optimisation in multi-access edge computing [21], while sustainable serverless edge scheduling distinguishes energy efficiency from carbon awareness [23]. Birkmose et al. [3] demonstrate on-device Home Assistant intent detection, slot filling, and JSON calls. HomeLLaMA [7] explores personalised assistance, and CIDER [8] combines small models with an ontology. AIoT Smart Home [16] studies autonomous device-control agents, while SmartIntent [20] integrates compact models into serverless building automation. *LocalIntent* instead examines backend-facing selection and validity across commands, persistent rules, clarifications, and rejections.

AutoIoT [18] and GPIoT [17] investigate IoT program synthesis. Almond [5] maps language to executable ThingTalk programs. RuleBot++ [6] creates and modifies Home Assistant rules, and Ply [1] supports inspection and user control in trigger–action programming. Everyday-automation studies highlight challenges in object/service selection, timing, and expressiveness [14]. These approaches primarily address program or rule construction rather than the preceding choice between execution, rule creation, clarification, and rejection.

Decomposed Prompting [9] separates complex tasks into stages. Schema Reinforcement Learning [12], SLOT [19], and format comparisons [22] address structured-output compliance, while constrained decoding [2] can enforce formal languages at potential efficiency or accuracy costs. Yet a valid structure need not imply correct outcome selection or support for generated entities, actions, and parameters. LLM-based parameter collection for energy optimisation [15] and feedback-based refinement [13] address related issues. *LocalIntent* integrates outcome selection, branch-specific generation, capability validation, and optional repair. We evaluate complete designs, not isolated mechanisms.

3 LocalIntent System Architecture

The system comprises three layers: access receives requests; the core manages context, routing, generation, and validation; the device layer exposes device and sensor services (Figure 1). The prototype implements the Internal Gateway, Intent Engine, Automation Engine, Context Aggregator, and State Storage. The Action Dispatcher and Automation Engine are intended to receive validated commands and rules. Experiments assess outputs and latency without physical actuation or concurrent state management.

Formally, for request u at time t, let $C_t = \langle Z, D, S, X_t, \mathcal{K} \rangle$ denote the local context. Z, D, and S are configured location zones, actionable devices, and observable sensors. X_t is the record's state snapshot, and registry $\mathcal{K}$ associates entities with identifiers, types, zones, and supported capabilities. For devices, it records actions and parameter

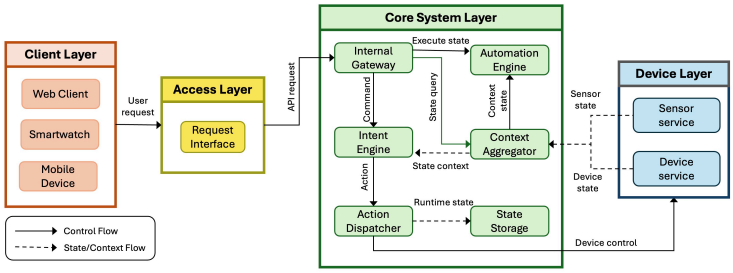


Fig. 1. *LocalIntent* within a layered smart building automation platform.

definitions, including types, units, ranges, or admissible values. For sensors, it records observable attributes and supported trigger predicates. D and S may overlap.

Let $\mathcal{P} = \{\textsc{Command}, \textsc{Rule}, \textsc{Clarification}, \textsc{Rejection}\}$ be the outcomes. Each outcome has specification $\Sigma_p = \langle \mathcal{S}_p, \Phi_p \rangle$, where $\mathcal{S}_p$ is its parser-accepted structure and Φ_p its semantic constraints. A Command is an immediate target–action–parameter operation; a Rule is a persistent trigger–action structure; Clarification requests information needed for a supported completion; and Rejection denotes functionality for which no supported completion exists within the registered capabilities or platform scope. *LocalIntent* routes over the coarser set $\mathcal{H} = \{\textsc{Ordinary}, \textsc{Rule}\}$, where Ordinary produces a command, clarification, or rejection. With mapping $\pi(\textsc{Rule}) = \textsc{Rule}$ and $\pi(p) = \textsc{Ordinary}$ otherwise, the branch specification is the collection of admissible outcome alternatives, $\Sigma_h = \{\Sigma_p : p \in \mathcal{P},\ \pi(p) = h\}$. Thus, the Rule branch contains only the Rule outcome specification, whereas the Ordinary branch contains the Command, Clarification, and Rejection specifications.

LocalIntent separates routing, generation, and validation:

$$h = \mathcal{R}(u, C_t), h \in \mathcal{H}, r = \mathcal{G}_h(u, C_t), (\hat{y}, e_p) = \text{ParseNorm}(r), \mathcal{V}_h(\hat{y}, C_t, \Sigma_h) \to (b, e).$$

$\mathcal{R}$ selects a branch, $\mathcal{G}_h$ produces raw response r, and ParseNorm parses and normalises it into candidate $\hat{y}$ or parse diagnostics e_p. Deterministic validator $\mathcal{V}_h$ then applies the parsing-independent structural, capability, parameter, and rule-consistency checks applicable to the generated outcome. $b \in \{0, 1\}$ indicates success and e contains structured diagnostics. Validated commands and rules are intended for the Action Dispatcher and Automation Engine, respectively; clarifications and rejections return via the access layer.

The *LocalIntent+SemanticRepair* variant permits at most one additional LLM call after validation failure. Its prompt contains the request, context, raw and normalised outputs, validation diagnostics, and diagnostic-specific guidance. The candidate must pass the same validator and a reference-free minimal-change guard. The guard preserves the outcome family and device/action identities unless the diagnostic permits reconstruction. Missing-parameter failures may complete required fields while preserving the normalised semantic payload. The candidate replaces the initial output only if it passes both checks. Otherwise, the initial output is retained only for paired experimental scoring and remains ineligible for downstream hand-off if validator-rejected.

4 Outcome-Specific Structured Generation

LocalIntent uses one local LLM call for ordinary/rule routing and another for branch generation. "Outcome-specific" denotes the distinct output alternatives and validity conditions embedded within these two branch prompts, not four separate generation calls. For compactness, $\mathcal{S}_h$ and Φ_h below denote the structures and constraints induced by the outcome specifications collected in Σ_h. Their mechanically evaluable subset is checked after generation.

4.1 Outcome Routing

For a request u and automation context C_t, the routing call predicts $\hat{h} = \mathcal{R}_\theta(u, C_t)$, $\hat{h} \in \{\textsc{Ordinary}, \textsc{Rule}\}$, where $\mathcal{R}_\theta$ is the routing call to model θ.

If the *LocalIntent* gate does not return a valid label, a predefined temporal-cue heuristic selects Ordinary or Rule. This heuristic recovers only a temporal branch label. It does not independently re-establish completeness, capability support, or representability and can therefore violate the normal Rule-eligibility policy.

Reference outcomes follow the policy:

$$p^* = \begin{cases} \textsc{Rejection}, & \text{if the requested functionality has no supported completion,} \\ \textsc{Clarification}, & \text{if a supported completion exists but required information is unresolved,} \\ \textsc{Rule}, & \text{if the request is complete, supported, persistent, and representable,} \\ \textsc{Command}, & \text{if the request is complete, supported, and immediate.} \end{cases}$$

The reference branch is $h^* = \pi(p^*)$. Accordingly, branch routing is correct when $\hat{h} = h^*$, although branch-routing accuracy is not reported as a separate aggregate metric.

A supported completion uses registered entities, actions, and valid parameters and is representable in the appropriate output schema without changing the requested functionality. Missing information therefore leads to Clarification only when such a completion remains possible. Unavailable functionality leads to Rejection. Temporal cues such as *"when"*, *"whenever"*, and *"if"* may indicate persistence, but rule selection also requires a representable trigger and supported action. An imperative with an unresolved target requires clarification, not execution.

4.2 Outcome-Specific Generation

After routing, *LocalIntent* assembles a prompt for the predicted branch: $q_{\hat{h}} = q_{\text{shared}} \oplus u \oplus \text{ctx}_{\hat{h}}(C_t) \oplus \mathcal{S}_{\hat{h}} \oplus \Phi_{\hat{h}}$, where $\oplus$ denotes deterministic assembly, q_{shared} defines the task, $\text{ctx}_{\hat{h}}$ serialises branch-relevant context, $\mathcal{S}_{\hat{h}}$ defines structure, and $\Phi_{\hat{h}}$ supplies constraints.

The two-branch design separates eligible persistent rules from ordinary requests, which retain commands, clarifications, and rejections. This delays the ordinary outcome decision until generation can apply capability and completeness constraints, while restricting rule generation to trigger–action structures. Such conditioning aims to reduce incompatible outputs but does not guarantee correct routing or semantic accuracy.

The ordinary branch receives zones, devices, supported actions, parameter definitions, relevant states, and the sensor/predicate capability information needed to assess

potentially persistent requests. It produces commands, clarifications, or rejections within their respective guardrails. The rule branch receives the full trigger-relevant sensor and predicate context and must produce exactly one trigger and one action. Capability information enables the ordinary branch to distinguish unresolved from unsupported persistent requests, while the rule branch uses it to ground the trigger and action in registered capabilities. For a parseable output, the generated backend outcome is

$$\hat{p} = \begin{cases} \textsc{Rule}, & \hat{h} = \textsc{Rule}, \\ \mathrm{Outcome}(\hat{y}), & \hat{h} = \textsc{Ordinary}, \end{cases}$$

where $\mathrm{Outcome}(\hat{y})$ reads the command, clarification, or rejection discriminator from the normalised Ordinary-branch output.

In the rule schema, a trigger contains a sensor, comparison operator, and threshold. Its action contains a target, supported action, and required parameters. Clarification and rejection are explicitly non-executable: the former requests information that could yield a valid output, while the latter identifies functionality unavailable under current capabilities. Neither includes a command or rule payload. Table 1 groups parser-accepted fields by semantic role rather than reproducing literal implementation field names.

4.3 Validation Interface

The deterministic validator checks structure, registered entities, supported actions, parameter domains, prohibited fields, and mechanically testable rule consistency. Whether an output correctly expresses the user's goal is assessed separately against reference annotations. Validation failures prevent downstream hand-off (Algorithm 1).

5 Performance Evaluation

We compare *LocalIntent* with *Monolithic* and *TwoStageGeneric* using six dataset-specific diagnostic metrics, paired repair analysis, and local inference latency.

LocalIntent and *TwoStageGeneric* share models, requests, capability snapshots, parsing, validation, and scoring, but differ in routing, prompts, output organisation, context presentation, and fallbacks. *TwoStageGeneric* receives $\mathcal{S}_p$ without LocalIntent-specific

Table 1. Outcome-specific fields and generation constraints.

Outcome	Semantic fields	Generation guidance
Command	target, action, parameters	Preserve immediate semantics. Select a registered target and supported action. Provide the required valid parameters. Do not introduce rule fields or infer unresolved information.
Rule	predicate-bearing trigger, action	Preserve persistent semantics. Map the trigger to a registered sensor and supported predicate. Select a supported action. Maintain consistency between the trigger predicate and action. Do not represent the action as immediate execution.
Clarification	unresolved element, question	Identify the information preventing an otherwise supported completion. Request only the information needed to continue. Do not generate command or rule fields.
Rejection	unsupported element, reason	Identify the requested functionality that is unavailable under the registered capabilities or platform scope. Do not generate command or rule fields.

Algorithm 1 Branch-specific structured generation in *LocalIntent*

Require: Request u, local context C_t
Ensure: Outcome $\hat{p}$ and candidate $\hat{y}$ when parseable, status b, diagnostics e
$\hat{h} \leftarrow \mathcal{R}_\theta(u, C_t)$
if $\hat{h} \notin \{\textsc{Ordinary}, \textsc{Rule}\}$ **then**
  $\hat{h} \leftarrow \text{TemporalFallback}(u)$
Construct $q_{\hat{h}}$ from u, branch-relevant context, $\mathcal{S}_{\hat{h}}$, and $\Phi_{\hat{h}}$
$r \leftarrow \text{LLM}_\theta(q_{\hat{h}})$
$(\hat{y}, e_p) \leftarrow \text{ParseNorm}(r)$
if $e_p \neq \varnothing$ **then**
  $\hat{p} \leftarrow \varnothing$; $\hat{y} \leftarrow \varnothing$; $b \leftarrow 0$; $e \leftarrow e_p$
else
  $\hat{p} \leftarrow \textsc{Rule}$ if $\hat{h} = \textsc{Rule}$; otherwise $\hat{p} \leftarrow \text{Outcome}(\hat{y})$
  $(b, e) \leftarrow \mathcal{V}_{\hat{h}}(\hat{y}, C_t, \Sigma_{\hat{h}})$
if $b = 0$ **then**
  Mark the request as not ready for downstream hand-off
return $(\hat{p}, \hat{y}, b, e)$

routing guidance or semantic constraints Φ_p. Invalid *TwoStageGeneric* routes default to Clarification, whereas *LocalIntent* uses a temporal-cue fallback. Thus, comparisons assess complete designs rather than individual mechanisms.

5.1 Evaluation Design

RQ1 compares *Monolithic*, *TwoStageGeneric*, and *LocalIntent* on reference-based correctness. *RQ2* examines differences between non-persistent requests and rules, while *RQ3* evaluates local inference latency. *Monolithic* selects and generates an outcome in one call; *TwoStageGeneric* performs four-way routing, and *LocalIntent* uses ordinary/rule routing. Paired initial and final outputs assess *LocalIntent+SemanticRepair*. Generation settings are shared across methods for each model.

5.2 Datasets and Local Models

Each JSONL record contains an instruction, device and sensor state, timestamp, and reference output. The non-persistent dataset has 101 requests: 54 commands, 31 clarifications, and 16 rejection cases. Its references use `needsClarification`, `message`, and `result`. Rule references contain one predicate-bearing trigger and one action within a `rule` object. The rule dataset contains 54 records representing 49 distinct instruction–state pairs. Five pairs have two reference actions subsequently judged admissible. All 54 records are retained, giving these five pairs double weight. Reference-based metrics do not exhaustively establish semantic validity. The dataset covers six trigger-sensor and five action-device types using >= or <=, state-derived thresholds, supported actions, and legal parameters. It contains only clear executable rules and does not test clarification or rejection of rule requests.

Written specifications defined task boundaries, capabilities, value ranges, schemas, and consistency constraints. Candidate records were manually checked for task type, state consistency, logical validity, and JSON syntax. LLM-assisted checks were advisory, and reported issues were manually adjudicated before correction or removal. Because

prompt exemplars overlap some benchmark formulations, results measure controlled prompt-development performance rather than held-out generalisation.

Outputs were generated with Ollama 0.32.1 on an Apple M4 MacBook Air with 16 GB memory using `gemma2:2b`, `qwen2.5:3b`, `gemma3:4b`, and `gemma2:9b`. We used a 4096-token context window and a 120 s timeout. Temperature, top-p, top-k, and seed were not explicitly set and followed Ollama or model defaults. Models remained loaded in memory during evaluation.

5.3 Metrics

Each normalised output receives six binary, dataset-specific diagnostic scores. For non-persistent requests, intent understanding (IU) requires an exact device–action match. For rules, it requires the reference trigger sensor, action device, and action, but not operator, value, or parameters. Device identification (DI) matches action-device sets and excludes rule-trigger sensors. Clarification handling (CH) matches `needsClarification` for non-persistent requests and rule/non-rule mode for rules. Action/parameter correctness (APC) requires matching actions and parameters, with tolerances of ±1 for temperature, brightness, level, and channel and ±5 for volume and `openPercentage`. Categorical values require exact agreement. Rule APC also requires an exact trigger object. Output-format compliance (OFC) checks normalised schema, registered identifiers, supported actions, and parameter keys but not agreement with the reference outcome. Language consistency (LC) checks message language for non-persistent outputs but only the top-level rule structure. Clarification/rejection message quality is unmeasured.

Except for one *LocalIntent+SemanticRepair* output for which no metric record exists, missing or unparsable outputs receive zero on all metrics. Rates are mean binary scores over records within each dataset. Metrics overlap, and CH and LC change meaning across datasets, so results are stratified and the metrics are not interpreted as independent dimensions. Repair is assessed through paired metric changes; Figure 7 reports repair-call counts. Latency uses observed mean, median, and 95th percentile.

5.4 Baseline Comparison

LocalIntent records higher pooled IU, DI, CH, and APC than *TwoStageGeneric* on both datasets, with larger gains on rules. RQ1–RQ2 concern the three primary methods; Figure 2 presents all methods across four Qwen and Gemma models and both datasets.

On non-persistent requests, pooled IU, DI, CH, and APC are 0.396, 0.421, 0.579, and 0.307 for *TwoStageGeneric*, and 0.532, 0.550, 0.621, and 0.428 for *LocalIntent*, each across 404 model–record evaluations. *LocalIntent+SemanticRepair* obtains 0.546, 0.563, 0.640, and 0.494 across 403 available evaluations because one output lacks a metric record. Both *LocalIntent* conditions exceed *TwoStageGeneric* on all four pooled metrics; the repair condition also has higher pooled rates than the base condition. Both conditions exceed *TwoStageGeneric* in IU, DI, and APC for all four models. For *LocalIntent*, CH is higher for two models and lower for two; the repair condition exceeds *TwoStageGeneric* in CH for two models. OFC and LC remain near the ceiling in most non-persistent conditions, offering less discrimination than the semantic metrics.

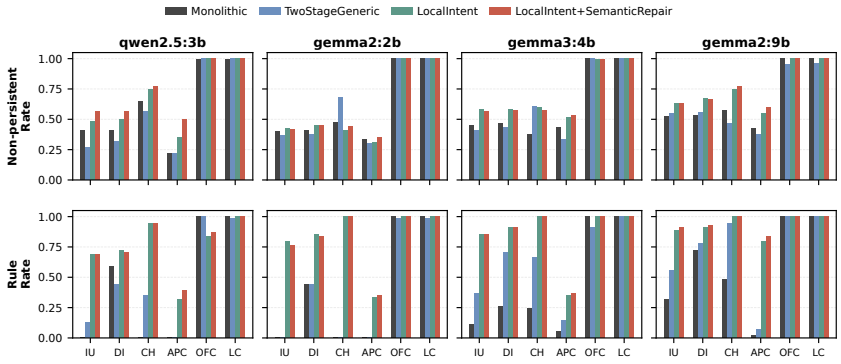


Fig. 2. Six dataset-specific metrics by method and model.

On rules, pooled IU, DI, CH, and APC are 0.264, 0.593, 0.491, and 0.056 for *TwoStageGeneric*; 0.806, 0.847, 0.986, and 0.449 for *LocalIntent*; and 0.801, 0.843, 0.986, and 0.486 for *LocalIntent+SemanticRepair*, each across 216 model–record evaluations. *LocalIntent* exceeds *TwoStageGeneric* on all four metrics for every model. Compared descriptively with the separate *LocalIntent* condition, the repair condition has slightly lower pooled IU and DI, identical CH, and higher APC. Most gains over *TwoStageGeneric* are therefore already present in base *LocalIntent*.

In the separately generated *LocalIntent* condition, 511 of 620 outputs passed deterministic validation, but 277 of those failed at least one reference-based metric. Validator acceptance therefore did not establish full reference agreement.

5.5 Validation-Guided Repair

Figure 3 compares paired initial (blue) and selected final (red) outputs under *LocalIntent+SemanticRepair*. Across all eight model–dataset combinations, final APC is no lower than initial APC, while paired IU, DI, CH, OFC, and LC totals remain unchanged. Net gains in APC-correct evaluations are +13 for `qwen2.5:3b`, +2 for `gemma2:2b`, +6 for `gemma3:4b`, and +6 for `gemma2:9b`, totalling 27. Most gains occur on non-persistent requests; rules show smaller positive gains for all four models.

5.6 Local Inference Latency

Observed local inference latency varies with model, request type, and summary statistic across *LocalIntent*, *TwoStageGeneric*, and the repair variant.

Observed P95 end-to-end latency shows no uniform advantage for either *LocalIntent* or *TwoStageGeneric* (Figure 4). On non-persistent requests, *LocalIntent* has lower P95 latency for `gemma3:4b` and `gemma2:9b`, slightly lower latency for `gemma2:2b`, and higher latency for `qwen2.5:3b`. On rules, it is lower for `qwen2.5:3b` and `gemma3:4b`, but higher for `gemma2:2b` and `gemma2:9b`. *LocalIntent+SemanticRepair* generally has higher P95 latency than *LocalIntent*, with the largest increase for non-persistent `gemma2:9b` requests.

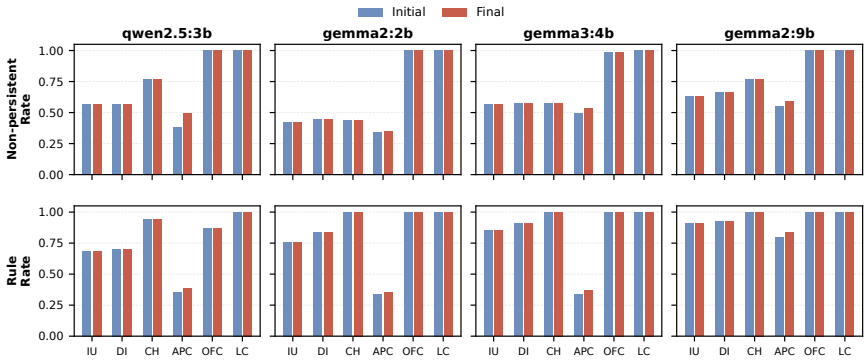


Fig. 3. Paired initial/final repair metrics; rejected repairs retain initial outputs.

Mean end-to-end latency favours *LocalIntent* over *TwoStageGeneric* for three of four models on non-persistent requests, with the largest reductions for `gemma3:4b` and `gemma2:9b` (Figure 5). On rules, it is faster only for `gemma3:4b`. The repair variant has higher mean latency than *LocalIntent* in all eight model–dataset combinations, with the largest increase for non-persistent `gemma2:9b` requests (29.60 versus 22.26 s).

Median latency follows similar model-dependent trends to the mean while being less sensitive to long-running requests (Figure 6). For non-persistent requests, *LocalIntent* has lower medians than *TwoStageGeneric* for all three Gemma models, with nearly identical values for `qwen2.5:3b`. On rules, *LocalIntent* has a lower median only for `gemma3:4b`. Repair increases median latency relative to *LocalIntent* in six of eight combinations; the exceptions are non-persistent `gemma3:4b` and rule-based `qwen2.5:3b`.

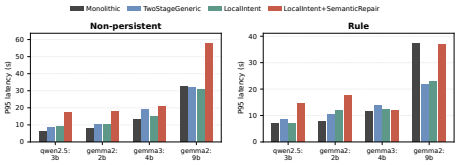


Fig. 4. Observed end-to-end P95 latency by method, model, and dataset.

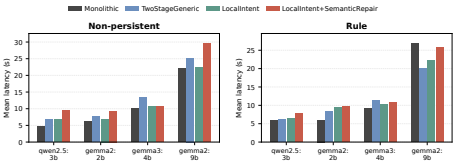


Fig. 5. Observed end-to-end mean latency by method, model, and dataset.

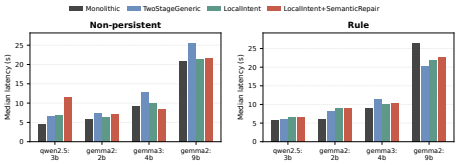


Fig. 6. Observed end-to-end median latency by method, model, and dataset.

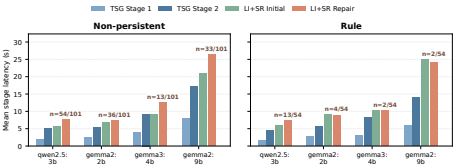


Fig. 7. Model-call latency by stage; purposes and counts vary.

Generation dominates routing latency for *TwoStageGeneric* across models and datasets (Figure 7). For non-persistent `gemma2:9b` requests, recorded routing and generation times are 8.05 and 17.17 s, respectively. Under *LocalIntent+SemanticRepair*, initial processing and triggered repair calls are the main measured components. Repair-call latency varies substantially across models and datasets and is particularly high for `gemma2:9b`, reaching 26.51 s on non-persistent requests. These measurements help explain the end-to-end trends but remain descriptive rather than controlled component comparisons because stage purposes and invocation counts differ across methods.

6 Conclusion

This paper examined branch-specific structured generation for locally executed smart building automation. *LocalIntent* separates ordinary requests from eligible persistent-rule requests, producing commands, clarifications, or rejections in the ordinary branch and predicate-bearing trigger–action structures in the rule branch. On the evaluated benchmark, the complete design shows its largest correctness gains over generic two-stage prompting for persistent rules, where the baseline frequently fails to produce matching rule-mode outputs. Non-persistent differences are smaller and model-dependent, while paired repair analysis shows targeted action/parameter-correctness (APC) improvements without reductions in the other aggregate metric totals. These descriptive results cover four models, one building configuration, and one generation per method–model–record. Because routing, prompts, schemas, context, and fallbacks change together, component contributions cannot be isolated. Future work should examine held-out multi-building requests, generation stability, targeted ablations, and constrained-decoding baselines. Remaining semantic errors underscore the central lesson: structural and capability validation alone cannot ensure semantic correctness or backend readiness.

References

1. Aveni, T.J., Mor, H., Fox, A., Hartmann, B.: Generative trigger-action programming with ply. In: Proc. of the 38th Annual ACM Symp. on User Interface Software and Technology. ACM, New York, NY, USA (2025)
2. Beurer-Kellner, L., Fischer, M., Vechev, M.: Guiding LLMs the right way: fast, non-invasive constrained generation. In: Proc. of the 41st Int. Conf. on Machine Learning. pp. 3658–3673 (2024)
3. Birkmose, R., Reece, N.M., Norvin, E.H., Bjerva, J., Zhang, M.: On-device LLMs for home assistant: Dual role in intent detection and response generation. In: Proc. of the 10th Workshop on Noisy and User-generated Text. pp. 57–67 (2025)
4. Calò, T., De Russis, L.: Enhancing smart home interaction through multimodal command disambiguation. Personal and Ubiquitous Computing **28**(6), 985–1000 (2024)
5. Campagna, G., Ramesh, R., Xu, S., Fischer, M., Lam, M.S.: Almond: The architecture of an open, crowdsourced, privacy-preserving, programmable virtual assistant. In: Proc. of the 26th Int. Conf. on World Wide Web. p. 341–350. WWW '17, Int. World Wide Web Conf. Steering Committee, Republic and Canton of Geneva, CHE (2017)
6. Gallo, S., Paternò, F., Malizia, A.: A conversational agent for creating automations exploiting large language models. Personal and Ubiquitous Comp. **28**(6), 931–946 (2024)

7. Huang, X., Shen, L., Ma, Z., Zheng, Y.: Toward privacy-preserving and personalized smart homes via tailored small language models. IEEE Trans. on Mobile Computing **25**(7), 9505–9520 (2026). https://doi.org/10.1109/TMC.2026.3654976
8. Jeong, D., Woo, H.: On-device intent reasoning for smart home agents via ontology-augmented sLLMs. IEEE Access **13**, 197645–197662 (2025)
9. Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., Sabharwal, A.: Decomposed prompting: A modular approach for solving complex tasks. In: The 11th Int. Conf. on Learning Representations (2023)
10. King, E., Yu, H., Lee, S., Julien, C.: Sasha: Creative goal-oriented reasoning in smart homes with large language models. Proc. ACM Interact. Mob. Wearable Ubiquitous Technol. **8**(1) (2024)
11. Li, S., Guo, Y., Yao, J., Liu, Z., Wang, H.: HomeBench: Evaluating LLMs in smart homes with valid and invalid instructions across single and multiple devices. In: Proc. of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 12230–12250 (2025)
12. Lu, Y., Li, H., Cong, X., Zhang, Z., Wu, Y., Lin, Y., Liu, Z., Liu, F., Sun, M.: Learning to generate structured output with schema reinforcement learning. In: Proc. of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 4905–4918 (2025)
13. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al.: Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems **36**, 46534–46594 (2023)
14. Mattioli, A., Paternò, F.: Personalised recommendations for daily automations controlled by mobile augmented reality. Proc. ACM Hum.-Comput. Interact. **9**(4) (2025)
15. Michelon, F., Zhou, Y., Morstyn, T.: Large language model interface for home energy management systems. In: Proc. of the 16th ACM Int. Conf. on Future and Sustainable Energy Systems. pp. 590–602 (2025)
16. Rivkin, D., Hogan, F., Feriani, A., Konar, A., Sigal, A., Liu, X., Dudek, G.: AIoT smart home via autonomous LLM agents. IEEE Internet of Things Journal **12**(3), 2458–2472 (2025)
17. Shen, L., Yang, Q., Huang, X., Ma, Z., Zheng, Y.: GpIoT: Tailoring small language models for IoT program synthesis and development. In: Proc. of the 23rd ACM Conf. on Embedded Networked Sensor Systems. pp. 199–212 (2025)
18. Shen, L., Yang, Q., Zheng, Y., Li, M.: AutoIoT: LLM-driven automated natural language programming for AIoT applications. In: Proc. of the 31st Annual Int. Conf. on Mobile Computing and Networking. pp. 468–482 (2025)
19. Shen, Z., Wang, D.Y.B., Mishra, S.S., Xu, Z., Teng, Y., Ding, H.: SLOT: Structuring the output of large language models. In: Potdar, S., Rojas-Barahona, L., Montella, S. (eds.) Proc. of the Conf. on Empirical Methods in Natural Language Processing: Industry Track. pp. 472–491. Association for Computational Linguistics, Suzhou (China) (2025)
20. Sun, S., Chen, Z., Shi, D., Li, C., Wu, Z., Tabatabaee Malazi, H.: SmartIntent: A serverless LLM-Oriented architecture for intent-driven building automation. In: 16th IEEE Int. Conf. on Cloud Computing Technology and Science (CloudCom). pp. 1–8 (2025)
21. Tabatabaee Malazi, H., Clarke, S.: Distributed service placement and workload orchestration in a multi-access edge computing environment. In: 2021 IEEE Int. Conf. on Services Computing (SCC). pp. 241–251 (2021)
22. Yuan, H., Zhao, Y., Zhang, L., Luo, W., Ma, Z.: Quantifying the impact of structured output format on large language models through causal inference. In: Findings of the Association for Computational Linguistics: EACL 2026. pp. 1771–1795 (2026)
23. Zhu, S., Hurley, N., Tabatabaee Malazi, H.: Does energy-efficiency mean carbon-awareness? a critical evaluation of sustainable serverless edge scheduling. In: Proc. of the 18th IEEE/ACM Int. Conf. on Utility and Cloud Computing. ACM, New York, NY, USA (2025)